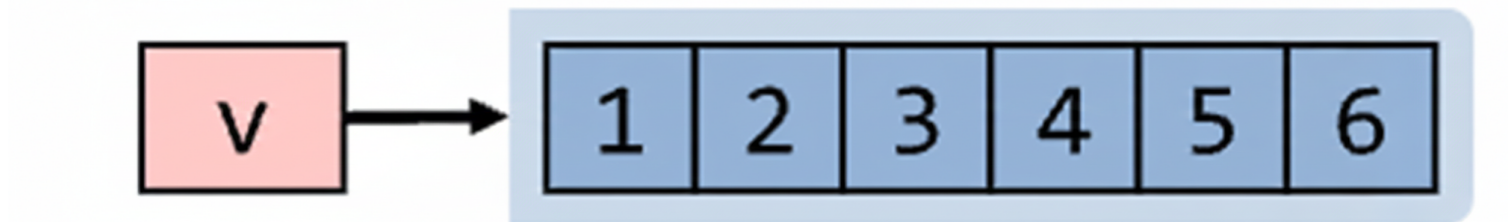


C++ QUICK REFERENCE

- Containers: store collections (vector, list, map, set, deque, ...)
- Iterators: traverse containers (input, forward, bidirectional, random-access)
- Algorithms: operate on iterator ranges (sort, find, for_each, ...)
- Functors: objects like functions, adapters (stack, queue)

std::vector

Dynamic contiguous array.



```
#include <vector>
#include <iostream>

int main() {
    std::vector<int> v;
    for (int i = 0; i < 5; ++i) v.push_back(i * 10);

    for (std::vector<int>::size_type i = 0; i < v.size(); ++i)
        std::cout << v[i] << " ";
    std::cout << '\n';

    for (std::vector<int>::iterator it = v.begin(); it != v.end(); ++it)
        std::cout << *it << " ";
    std::cout << '\n';
}
```

std::list

Doubly-linked list.

```
#include <list>
#include <iostream>

int main() {
    std::list<int> L;
    L.push_back(1); L.push_back(2); L.push_back(3);
    for (std::list<int>::const_iterator it = L.begin(); it != L.end(); ++it)
        std::cout << *it << " ";
}
```

Associative containers

```
#include <map>
#include <string>
#include <iostream>

int main() {
    std::map<std::string, int> ages;
    ages["Alice"] = 30;
    ages["Bob"] = 25;

    for (std::map<std::string, int>::const_iterator it = ages.begin(); it != ages.end(); ++it)
        std::cout << it->first << ": " << it->second << '\n';
}
```

Container adapters

```
#include <queue>
#include <iostream>

int main() {
    std::queue<int> q;
    q.push(1);
    q.push(2);
    q.push(3);

    while (!q.empty()) {
        std::cout << q.front() << " ";
        q.pop();
    }
    std::cout << '\n';
}
```

Unordered maps and pairs

```
#include <map>
#include <string>
#include <utility>
#include <iostream>

int main() {
    std::map<std::string, int> ages;
    ages.insert(std::make_pair(std::string("Alice"), 30));
}
```

```
ages.insert(std::make_pair(std::string("Bob"), 25));

for (std::map<std::string, int>::const_iterator it = ages.begin(); it != ages.end(); ++it)
    std::cout << it->first << ": " << it->second << '\n';

// If your implementation provides unordered_map:
/* #include <unordered_map>
   std::unordered_map<std::string, int> um;
   um["Alice"] = 30;
*/
}
```

Iterators

```
#include <vector>
#include <algorithm>
#include <iostream>

int main() {
    std::vector<int> v;
    v.push_back(3); v.push_back(1); v.push_back(2);
    std::sort(v.begin(), v.end());
    for (std::vector<int>::const_iterator it = v.begin(); it != v.end(); ++it)
        std::cout << *it << ' ';
}
```

Algorithms

Remove-Erase Idiom

```
#include <vector>
#include <algorithm>

int main() {
    std::vector<int> v;
    v.push_back(1); v.push_back(2); v.push_back(3); v.push_back(2);
    v.erase(std::remove(v.begin(), v.end(), 2), v.end());
}
```

Transform and Accumulate

```
#include <vector>
#include <algorithm>
#include <numeric>
#include <iostream>

struct Square { int operator()(int x) const { return x*x; } };

int main() {
    std::vector<int> v;
    v.push_back(1); v.push_back(2); v.push_back(3);
    std::vector<int> squares(v.size());
    std::transform(v.begin(), v.end(), squares.begin(), Square());
    int sum = std::accumulate(squares.begin(), squares.end(), 0);
    std::cout << "sum = " << sum << '\n';
}
```

Functors and Comparators

```
#include <string>
#include <vector>
#include <algorithm>

struct ByLength {
    bool operator()(const std::string& a, const std::string& b) const {
        return a.size() < b.size();
    }
};

int main() {
    std::vector<std::string> v;
    v.push_back("apple"); v.push_back("kiwi"); v.push_back("banana");
    std::sort(v.begin(), v.end(), ByLength());
}
```

Useful Idioms

- Use reserve() on vector when size is known.
- Pass large containers by const&.
- Use lower_bound / upper_bound.

```
#include <vector>
#include <algorithm>
#include <iostream>
#include <numeric>
#include <map>
#include <string>

struct SquareFunctor { int operator()(int x) const { return x * x; } };

int main() {
    std::vector<int> v;
    for (int i = 1; i <= 5; ++i) v.push_back(i);

    std::vector<int> sq(v.size());
    std::transform(v.begin(), v.end(), sq.begin(), SquareFunctor());

    int sum = std::accumulate(sq.begin(), sq.end(), 0);
    std::cout << "sum of squares = " << sum << '\n';

    std::vector<std::string> words;
    words.push_back("apple");
    words.push_back("kiwi");
    words.push_back("banana");
    std::map<int, int> freq;
    for (std::vector<std::string>::const_iterator it = words.begin(); it != words.end(); ++it)
        ++freq[it->size()];

    for (std::map<int, int>::const_iterator it = freq.begin(); it != freq.end(); ++it)
        std::cout << "length " << it->first << ": " << it->second << " word(s)\n";
}
```

C++11

Type inference: ‘auto’, ‘decltype’

```
#include <vector>
#include <iostream>

int main() {
    std::vector<int> v = {1,2,3,4};
    for (auto it = v.begin(); it != v.end(); ++it)
        std::cout << *it << ' ';
    std::cout << "\n";

    auto sum = 0; // int
    decltype(sum) total = 0; // same type as sum
}
```

Range-based for

```
#include <vector>
#include <iostream>

int main() {
    std::vector<int> v = {10,20,30};
    for (auto x : v) std::cout << x << ' ';
    std::cout << '\n';
    for (auto &x : v) x += 1; // modify in-place
}
```

Lambda expressions

```
#include <algorithm>
#include <vector>
#include <iostream>

int main() {
    std::vector<int> v = {3,1,2};
    std::sort(v.begin(), v.end(), [](int a, int b){ return a<b; });
    std::for_each(v.begin(), v.end(), [](int x){ std::cout << x << ' '; });
    std::cout << '\n';
}
```

Move Semantics in C++

Beginning with C++11, the C++ language introduced *move semantics*, a mechanism enabling efficient transfer of resources from one object to another. Move semantics reduce

unnecessary copying of objects that manage heavy resources such as memory buffers, strings, or file handles.

We provide a detailed and rigorous explanation of move semantics, beginning with value categories and progressing to move constructors, move assignment, and usage recommendations.

Value Categories and Reference Types

Understanding move semantics requires understanding how C++ distinguishes persistent objects from temporary ones.

Lvalues

An **lvalue** refers to an object with a persistent memory location.

```
int x = 10; // x is an lvalue
```

Lvalues bind to lvalue references:

```
T&
```

Rvalues

An **rvalue** is a temporary expression without a stable address.

```
10 // rvalue literal
x + 5 // rvalue
std::string("hi") // temporary
```

Rvalues bind to rvalue references:

```
T&&
```

Temporaries may be safely moved from, enabling efficient resource transfers.

Move Constructors and Move Assignment

Classes that manage resources must implement:

- Move constructor
- Move assignment operator

Example Class

```
class MyString {
private:
    char* data;

public:
    // Constructor
    MyString(const char* s) {
        size_t n = std::strlen(s) + 1;
        data = new char[n];
        std::memcpy(data, s, n);
    }

    // Destructor
    ~MyString() {
        delete[] data;
    }

    // Copy constructor
    MyString(const MyString& other) {
        size_t n = std::strlen(other.data) + 1;
        data = new char[n];
        std::memcpy(data, other.data, n);
    }
}
```

Move Constructor

```
MyString(MyString&& other) noexcept : data(other.data) {
    other.data = nullptr;
}
```

Move Assignment Operator

```
MyString& operator=(MyString&& other) noexcept {
    if (this != &other) {
        delete[] data;
        data = other.data;
        other.data = nullptr;
    }
    return *this;
};
```